PROJECT RESILIENCE-ALPHA

Dossier d'ingénierie et de gouvernance
Cyber-résilience et reprise autonome

Workshop National EPSI / myDiL — BAC+5

Équipe Workshop2026-M2-G<n> · Session Octobre 2026

Démonstrateur à données entièrement fictives

1. Contexte et objectif

À l'automne 2026, une plateforme de paiement exploitée par une entreprise financière fictive subit une série d'incidents qui fragilisent la continuité de son activité. Les remises en service sont aujourd'hui manuelles, longues et difficilement reproductibles. L'initiative **Project Resilience-Alpha** vise à démontrer, sur un périmètre maîtrisé, qu'une reprise automatique peut être conçue, observée et vérifiée.

Ce dossier présente les décisions qui structurent notre démonstrateur : architecture et flux, rôle des composants, nature de l'incident, mécanismes de détection et de reprise, résultats des essais, éléments de sécurité, analyse des risques, PCA/PRA et choix de gouvernance. L'objectif n'est pas de livrer une plateforme industrielle, mais une **chaîne de cyber-résilience crédible**, exécutable et défendable techniquement.

Objectif démontré

- Un SI de paiement fictif, conteneurisé, exécute de vraies transactions simulées.
- Un incident de sécurité contrôlé est provoqué depuis un poste distinct (PC Attaquant).
- La plateforme détecte seule l'anomalie, décide d'une remédiation, l'exécute et **vérifie** le retour à un état utilisable.
- L'état du service est rendu visible avant, pendant et après l'incident.

2. Architecture et flux

La solution réunit les cinq domaines obligatoires du sujet au sein d'une plateforme unique, orchestrée par Docker Compose. Un seul point d'entrée — le WAF — est exposé ; tous les autres services communiquent sur un réseau interne isolé.

Couche	Composants	Rôle
Entrée	WAF (nginx)	Seul point exposé : filtrage applicatif, limitation de débit
Métier	payment-api (FastAPI), auth (JWT), PostgreSQL, vitrine	Comptes, transactions, authentification, interface
Observabilité	Prometheus, Grafana, Loki, Promtail	Métriques, visualisation, journaux centralisés (SIEM)
IA / Data	ml-detection, rag-assistant (Ollama+LangChain)	Détection d'anomalies ; assistant opérateur documentaire
Résilience	remediation, dashboard React/Tailwind	Reprise autonome et supervision temps réel

Réseau : **resilience_net** (interne Docker). Base, secrets et interfaces de gestion ne sont jamais publiés sur le réseau pédagogique.

3. Les cinq domaines de responsabilité

3.1 — Développement

Le SI de paiement est composé de microservices conteneurisés. L'**API métier** (FastAPI) crée les comptes et exécute les transactions de façon atomique (débit/crédit sous verrou, cohérence ACID). Le **service d'authentification** émet et vérifie des jetons JWT courts. La **vitrine** et le **dashboard** sont développés en React avec Tailwind CSS, conformément au pilier DEV.

3.2 — Infrastructure

La plateforme se déploie de manière reproductible via **Docker Compose** ; des playbooks **Ansible** automatisent les opérations (déploiement, redémarrage ciblé, isolation). La stack d'observabilité **Prometheus / Grafana / Loki** (alimentée par Promtail) renseigne l'état des services et alimente réellement l'analyse de l'incident — pas d'Elastic Stack, par souci de charge mémoire sur le PC Hôte.

3.3 — IA / Data

La détection ne se réduit pas à des conditions codées en dur : un modèle de machine learning non supervisé (détecteur de nouveauté à covariance robuste, *EllipticEnvelope*) apprend la « forme » du comportement normal et signale les écarts (détaillé en §4). Un second composant, l'**assistant RAG** (Ollama + LangChain, modèle local < 8 Md de paramètres), aide l'opérateur à retrouver des informations dans les procédures et l'architecture — en lecture seule, **sans aucun droit privilégié** sur le système, et distinct du moteur de remédiation.

3.4 — Cybersécurité

Les dispositifs défensifs attendus sont en place : durcissement des composants et des accès (conteneurs non-root, capacités réduites, no-new-privileges), filtrage applicatif par **WAF**, suivi des événements dans un **SIEM** (Loki/Grafana intégré à la supervision), et gestion des secrets par **SOPS** (alternative Vault documentée). Une attention particulière est portée au moteur de remédiation, qui dispose de droits élevés : il n'expose aucune interface d'administration ouverte (§5).

3.5 — Gouvernance et transformation

La résilience ne se limite pas aux opérations techniques. Une analyse de risques **EBIOS Risk Manager** (§6), un **PCA/PRA** adapté (§7) et une lecture des enjeux **NIS2/DORA** (§8) justifient les choix : quels services sont critiques, quels risques sont retenus, et ce que coûte l'arrêt d'une fonction de paiement.

4. Détection d'anomalies (ML)

Principe. On apprend à quoi ressemble une « journée normale » de la plateforme à travers quatre indicateurs issus de Prometheus, puis on trace une enveloppe autour de cette zone normale. Toute mesure qui en sort nettement est signalée. Le seuil de décision n'est pas écrit à la main : il est **calibré par le modèle** sur les données normales (environ 0,5 % de faux positifs tolérés).

Feature	Signification	Ce qu'un écart révèle
req_rate	Requêtes par seconde	Débit anormal → possible déni de service
error_ratio	Part de réponses 5xx	Erreurs anormales → injection ou panne applicative
latency_p95	Latence au 95e percentile	Latence anormale → saturation / contention base
inflight	Requêtes en cours	Accumulation → service en difficulté

Pourquoi ce modèle. Un *Isolation Forest* a d'abord été testé, mais il manquait certains écarts extrêmes portés par une seule variable. Le détecteur à covariance robuste mesure la distance de Mahalanobis au nuage normal : il tient compte des corrélations entre indicateurs et reste fiable sur ces cas. Le choix a été validé par tests automatisés (cas normal, pic de débit, pic de latence, pic d'erreurs → tous correctement classés).

Explicabilité. Pour chaque alerte, le service indique le *driver principal* (la feature la plus hors-normale) et une phrase d'explication opérationnelle, puis notifie le moteur de remédiation. Le modèle est raccordé à la supervision et expose ses propres méta-métriques à Prometheus.

5. Reprise autonome (remédiation)

Le moteur de remédiation ferme la boucle : il reçoit l'incident, décide d'une action selon une **politique explicite**, l'exécute sur les conteneurs, puis **vérifie réellement** le retour à un état utilisable avant de clôturer.

```
Détection → Décision (politique selon le driver) → Action (restart / durcissement / isolation) → Vérification (health/deep + transaction fictive test) → Clôture ou nouvelle tentative (max 3)
```

Driver d'anomalie	Plan d'action	Intention
latency_p95 / base down	restart base → restart API	Lever la contention / reconnecter
req_rate (DoS)	restart API (+ durcissement débit)	Absorber le pic, vider la file
error_ratio	restart API + flag investigation	Rétablir ET ouvrir une investigation sécurité
inflight	restart API	Vider la file de requêtes

Reprise réelle, pas cosmétique. Une reprise n'est déclarée que si la vérification métier réussit : santé profonde (base joignable + comptes lisibles) **et** une nouvelle transaction fictive qui aboutit. Un conteneur qui redémarre ne suffit pas.

Garde-fou du composant privilégié. Le moteur agit via une **liste fermée** d'actions prédéfinies (pas de shell distant générique), toutes journalisées et idempotentes ; son jeton est à portée limitée ; il ne peut pas être piloté par l'assistant RAG. On ne distingue pas non plus restauration de disponibilité et correction d'une compromission : une erreur anormale déclenche, en parallèle de la reprise, un marquage pour investigation.

6. Scénario d'attaque — laboratoire contrôlé

L'incident est provoqué depuis un **PC Attaquant** distinct, strictement contre notre propre SI fictif. Toute action offensive est interdite sur les réseaux du campus, un autre groupe ou un service tiers. Les scripts refusent de s'exécuter contre une cible non privée et exigent une confirmation explicite du cadre de labo ; débit et durée sont plafonnés.

- **Déni de service applicatif contrôlé** : une rafale bornée de requêtes légitimes crée un pic observable. Effet attendu : le WAF limite (429), l'alerte TrafficSpike se déclenche, la détection signale req_rate, la remédiation agit.
- **Sonde d'injection bénigne** : quelques motifs d'injection classiques sont envoyés pour vérifier que le WAF bloque (403) et journalise. L'API (requêtes paramétrées) n'est de toute façon pas vulnérable : la sonde constate la défense, elle ne contourne rien.

Il ne s'agit pas d'un outil d'attaque réutilisable : aucun exploit n'est livré. L'objectif est pédagogique — produire un effet mesurable que nos propres mécanismes doivent repérer et corriger seuls.

7. PCA / PRA

Indicateur	Cible (démo)	Justification
RTO (délai de reprise)	< 60 s	Reprise automatique sans intervention humaine
RPO (perte de données)	~ 0	Base transactionnelle ACID, pas de perte en vol
Disponibilité visée	Démontrable sur le labo	Objectif pédagogique, non un SLA industriel

Le plan couvre l'indisponibilité API/base, le pic de trafic/DoS et les erreurs anormales. Il distingue explicitement **restaurer la disponibilité** (faire redémarrer le service) de **corriger une compromission** (investigation distincte). Le PRA est testé en direct (chaos engineering), et le Clip Chrono matérialise un cycle complet attaque → anomalie → reprise → service rétabli.

Risques subsistant après la reprise

- Une reprise de disponibilité ne garantit pas l'absence de compromission résiduelle (d'où le flag d'investigation).
- Dépendance au PC Hôte unique (pas de redondance matérielle en labo).
- Rejeu possible tant que la cause racine n'est pas corrigée.

8. Conformité — NIS2 et DORA

Démarche de conseil : distinguer ce qu'un prototype de quatre jours **peut démontrer** de ce qu'il ne peut qu'**esquisser**, plutôt qu'un rapport de conformité générique détaché de l'architecture.

Exigence	Dans le projet	Démontrable en 4 j ?
Gestion des risques TIC (NIS2 art.21 / DORA 1)	Analyse EBIOS RM, cartographie des biens	Oui
Détection et gestion d'incidents (DORA 2)	Détection ML, remédiation, journaux SIEM	Partiel (notif. réglementaire : non)
Tests de résilience opérationnelle (DORA 3)	Chaos engineering, test du PRA en direct	Oui
Continuité / gestion de crise (NIS2)	PCA/PRA, reprise automatique testée	Partiel
Risque lié aux tiers TIC (DORA 4)	Dépendances images / PC Hôte documentées	Esquissé
Notification d'incident (24 h / 72 h)	Procédure décrite	Non (hors prototype)

Recommandations à une direction

- Investir d'abord dans la détection + reprise automatique (valeur mesurable : RTO < 60 s).
- Séparer budgétairement remédiation (disponibilité) et investigation (compromission).
- Passer de SOPS à Vault en production (secrets dynamiques, audit).
- Mettre en place le reporting réglementaire d'incident, non démontrable par un prototype mais obligatoire en exploitation.

9. Essais et limites assumées

Essais réalisés. Tests automatisés de la détection (cas normal + trois types d'anomalies), de la validation des transactions (montants négatif/nul rejetés), de la récupération documentaire du RAG et des garde-fous Red Team (cible publique refusée, confirmation exigée) — tous au vert. Validation syntaxique des configurations Docker/Prometheus/Grafana.

Limites. Prototype pédagogique de quatre jours : pas de mTLS interne, secrets en SOPS plutôt que Vault dynamique, PC Hôte unique sans redondance, modèle de détection entraîné sur des observations constituées faute d'historique de production. Ces arbitrages sont documentés dans le dépôt (security/, docs/). Ce que nous revendiquons est une chaîne de cyber-résilience crédible, exécutable et défendable — pas une plateforme industrielle prête à la production.

Projet Resilience-Alpha — Dossier d'ingénierie et de gouvernance · Données et contexte métier entièrement fictifs · Équipe Workshop2026-M2-G<n> · EPSI/myDiL BAC+5, Octobre 2026.